Threshold Anti-Pattern Audit

HelixCode Test-Suite Env-Fragile Threshold Anti-Pattern Audit

Type: Task · **Status:** active · **Date:** 2026-06-24 · **Mode:** READ-ONLY discovery/ triage (§11.4.118) **Scope:** `helix_code/` inner Go module — `tests/`, `internal/`, `applications/` test files. **Trigger:** A §11.4.6 anti-pattern (hardcoded absolute 50MB heap cap, point-snapshot HeapAlloc that swung 8.7→127MB across hosts) was found+fixed in `tests/memory` GC-pressure test. This sweep maps SIBLING + SIMILAR env-fragile hardcoded-threshold assertions. **Evidence basis:** FACT = lines read directly. UNCONFIRMED = inferred, not run (no execution performed). `tests/memory/*` and `tests/security/*` NOT edited (other streams own them); `tests/memory` only READ.

Findings table

file:line	test	t
`tests/memory/ memory_test.go:140-143`	`TestMemory_LeakDetection_RepeatedRequests`	d 1 it
`tests/memory/ memory_test.go:195-197`	`TestMemory_LeakDetection_ConcurrentRequests`	d 2 c

file:line	test	t
tests/memory/ memory_test.go:258-260	TestMemory_LeakDetection_JSONParsing	d 1 1
tests/memory/ memory_test.go:780-782	TestMemory_ResourceCleanup_Contexts	d 1 c
tests/memory/ memory_test.go:381-383	TestMemory_Allocation_ConnectionPooling	d 5
tests/memory/ memory_test.go:328-330	TestMemory_Allocation_LargePayloads	d s
tests/memory/ memory_test.go:706-708	TestMemory_ResourceCleanup_Goroutines	g (2
internal/config/ advanced_config_test.go:698	TestAdvancedConfiguration_Performance	d V
internal/config/ advanced_config_test.go:713	TestAdvancedConfiguration_Performance	d 1

file:line	test	t
`internal/event/bus_test.go:180`	(async publish test)	d s
`internal/tools/web/ web_test.go:452`	(FetchMultiple concurrency test)	d c
`internal/discovery/ health_monitor_test.go:119`	(warm-up branch)	t 5 N
`internal/discovery/ integration_test.go:441`	`WaitForService` timeout test	e 5 ti
`internal/hooks/ shell_runner_test.go:49`	(timeout test)	e n
`tests/e2e/phase3/ production_validation_test.go:160`	(production validation)	d ti
`tests/e2e/phase3/ performance_test.go:517`	(throughput scalability)	a t

file:line	test	t
`tests/e2e/phase3/performance_test.go:518`	(same)	s
`tests/e2e/phase3/performance_test.go:519`	(same)	a
`tests/performance/benchmark_test.go:274`	`TestPerformance_*` (RPS gate)	r (1
`tests/performance/benchmark_test.go:~277`	same	s
`tests/performance/scenarios/runner_test.go:64`	`TestRunScenario_*` (harness-noise gate)	c (c
`tests/qa/qa_test.go:711`	`TestQA_BuildQuality_NoRaceConditions`	r <

Robust exemplars (NOT defects — reference patterns for fixing the HIGHs)

- `tests/memory/memory_test.go:390-525` `TestMemory_GCPressure_HighAllocationRate` — the **fixed** pattern: multi-sample post-GC live-heap, bounded-growth-across-run invariant (the comment at lines 500-512 documents the exact 23MB?125MB cross-host variance lesson). **This is the template for fixing the 4 HIGH point-snapshot tests.**

- `internal/llm/ensemble_stress_chaos_test.go:44-60` `settleGoroutines(baseline)` — poll-until-stable, "no GROWTH beyond small slack" goroutine-leak invariant (deterministic regardless of scheduler).
- `internal/worker/lifecycle_regression_test.go:35-55` `settleGoroutines(watchdog)` — poll-until-two-stable-samples goroutine invariant.
- `tests/integration/provider_integration_test.go:760-790` `testResourceLeakDetection` — `after ≤ baseline+5` relative-to-baseline goroutine invariant.
- `internal/cognee/performance_optimizer_gpu_*_test.go` (lines ~160-238, all 5 GPU vendors) — `elapsed < <configuredTimeout> + Ns` — latency bound RELATIVE to the code's own configured timeout, not an absolute literal. Robust.

Counts per severity (FACT — from rows above)

Severity	Count	Items
HIGH	4	memory_test.go:140 / :195 / :258 / :780 (point-snapshot HeapAlloc-delta vs absolute MB cap)
MEDIUM	8	memory_test.go:706 (goroutine fixed-sleep); advanced_config_test.go:698 / :713 (1s/500ms CPU caps); event/bus_test.go:180 (50ms); web_test.go:452 (50ms); e2e/phase3 production_validation:160 (2s), performance_test:517 (throughput floor) + :519 (2s avg); benchmark_test.go:274 (RPS<10 floor)
LOW	9	memory ConnectionPooling:381, LargePayloads:328; health_monitor:119; discovery integration:441; shell_runner:49; performance_test successRate:518; benchmark P95:277; scenarios cv:64; qa goroutine:711

(MEDIUM count = 9 distinct assertions across 8 sites; e2e/phase3/performance_test.go contributes both throughput-floor and avg-latency.)

Prioritized fix list (HIGH first)

1. **HIGH — `tests/memory/memory_test.go:140,195,258,780`** (4 tests): port the 3 leak-detection siblings + the context-cleanup test from point-snapshot-HeapAlloc-vs-absolute-MB to the multi-sample bounded-growth invariant already proven in the same file's `TestMemory_GCPressure_HighAllocationRate` (lines 390-525). Single highest-value fix — same root cause as the already-acknowledged 50MB flake, same blast radius (nondeterministic FAIL on normal hosts). **NOTE: `tests/memory` is owned by another active stream — coordinate before editing.**
2. **MEDIUM — `internal/event/bus_test.go:180` + `internal/tools/web/web_test.go:452`**: sub-100ms absolute latency caps are the most flake-prone of the MEDIUMs (smallest margin vs GC/scheduler jitter). Replace with semantic/ratio invariants.
3. **MEDIUM — `internal/config/advanced_config_test.go:698,713`**: 1s/500ms CPU-bound timing caps flake under `-race`/coverage (which the project runs via `make test-coverage`). Drop timing assertions from unit tests; move perf tracking to benchmarks.
4. **MEDIUM — `tests/e2e/phase3/performance_test.go:517` + `benchmark_test.go:274`**: throughput floors (`target×0.8`, `rps<10`) are host-bound; calibrate to a measured baseline or gate behind a perf-host tag.
5. **MEDIUM — `tests/memory/memory_test.go:706`**: swap fixed `time.Sleep` for the `settleGoroutines` poll helper (same-file/cross-package exemplars exist).
6. **MEDIUM — `tests/e2e/phase3/production_validation_test.go:160` + `performance_test.go:519`**: 2s absolute response-time SLAs — document as SLA + ensure warm-up, or make percentile-based.
7. **LOW**: no action required; several (LargePayloads ratio, cv-cap, GPU `timeout+N`, successRate, goroutine sanity) are already correct env-independent patterns — keep as reference exemplars.

Honest scope notes (§11.4.6)

- **No tests were executed** — all severities are static-analysis judgments of flake-*likelihood*, not observed flakes (except the memory point-snapshot class, whose cross-host variance is documented FACT in the file's own comment lines 500-512 and the originating stream's report).
- `tests/security/*` was excluded from edit but NOT separately swept for threshold patterns beyond the general grep (no hits surfaced in the throughput/latency/goroutine/CV sweeps).
- Sweep covered: absolute latency caps (`time.Since < const`), heap/alloc byte caps, throughput floors, goroutine/connection counts, CV/variance caps, dial/connect timeouts. The previously-fixed discovery 100ms dial timeout did not resurface (already fixed).
- UNCONFIRMED: there may be additional sub-100ms `time.Sleep`-then-assert timing assertions in benchmark-only (`Benchmark*`) functions not surfaced by the `assert.Less`/`Greater` grep; benchmarks are not pass/fail-gated so low priority.